CreditCompass: An Intelligent Credit Risk Scoring and Agentic Lending Decision Support System

Shibaditya Deb, Arohi Jadhav, Parth Tandelwade, Jay Patil
Team CognitiveLens

***Abstract*—Accurate borrower credit risk assessment remains a critical challenge in consumer lending. This paper presents CreditCompass, a two-milestone AI-driven platform developed by Team CognitiveLens that first employs supervised machine learning to produce real-time probability-of-default scores, and then extends those predictions into a LangGraph-based agentic assistant that generates structured, regulation-aware lending recommendations. The platform is built on a stack comprising Flask, XGBoost, LangGraph, OpenRouter (Mistral-7B free tier), and Tailwind CSS, and is publicly deployed on Render at https://creditcompass-fp7x.onrender.com/. Milestone 1 exposes a `/predict` endpoint that accepts ten borrower features and returns a default probability and risk classification via a trained XGBoost model. Milestone 2 exposes an `/assess` endpoint driven by a four-node LangGraph state graph that gracefully handles incomplete data through dataset-median imputation, retrieves local lending rules from a structured JSON knowledge base, calls Mistral-7B via the OpenRouter API, and produces a seven-section structured lending assessment report. Risk decisions follow three calibrated tiers: <40% probability (Low Risk, APPROVE), 40–70% (Medium Risk, REVIEW), and >70% (High Risk, REJECT). The complete application requires no login and is freely accessible to any user.**

***Index Terms*—Credit risk scoring, probability of default, XGBoost, agentic AI, LangGraph, OpenRouter, Mistral-7B, lending decision support, Flask, retrieval-augmented generation.**

I. Introduction

Consumer lending is one of the most risk-sensitive activities in modern finance. Every loan approval decision implicitly involves estimating the probability that a borrower will default—a task historically handled by expert-designed scorecards and rigid rule-based systems. While such systems are interpretable and auditable, they fall short in two important ways: they fail to capture complex, non-linear interactions between borrower attributes, and they provide no structured, reasoned recommendation of the kind a human loan officer would produce.

Machine learning methods—particularly ensemble models such as gradient-boosted trees—have demonstrated substantially superior predictive accuracy on historical loan data compared to classical scorecards [**?** **?**]. However, a probability score alone does not constitute a lending decision. Loan officers need to understand which features are driving a risk assessment, how the borrower compares to regulatory thresholds, what risk mitigations might be applied, and what legal disclaimers apply. Meeting these needs requires not just prediction, but structured reasoning.

The maturation of large language models (LLMs) and agentic AI orchestration frameworks opens a compelling path toward filling this gap. By coupling ML-derived risk signals with LLM reasoning grounded in a local lending rules knowledge base, it becomes possible to construct a decision support assistant that is simultaneously data-driven, explainable, and operationally useful.

This paper describes CreditCompass, a platform built by Team CognitiveLens across two project milestones. In Milestone 1 (mid-semester), we train an XGBoost classifier on a consumer credit risk dataset, expose it through a Flask `/predict` endpoint, and present predictions through an animated probability gauge UI built with Tailwind CSS. In Milestone 2 (end-semester), we extend the system into a full agentic pipeline using a four-node LangGraph state graph, a local JSON lending rules store, and Mistral-7B served via the OpenRouter free-tier API—deployed publicly on Render.

The primary contributions of this work are:

- A two-endpoint Flask API—`POST /predict` for instant ML scoring (all ten fields required) and `POST /assess` for full agentic assessment (all fields optional, missing ones handled via median imputation)—plus a `GET /health` liveness check.
- A four-node LangGraph `StateGraph` with explicit typed state management, a `check_data_node` that catalogues missing fields, median-based graceful degradation in the `predict_node`, a local JSON rules loader, and an automatic rule-based fallback when the LLM API is unavailable.
- A local JSON lending knowledge base (`data/lending_rules.json`) covering three risk tiers, debt ratio bands, and six regulatory references, used to ground LLM recommendations without requiring a vector database.
- A publicly hosted, no-login application at https://creditcompass-fp7x.onrender.com/ with an animated Tailwind CSS frontend that renders a seven-section AI lending assessment report with decision banner and missing-data alert.

II. Related Work

Statistical methods for credit risk prediction have a long history. **?**] applied linear discriminant analysis to financial ratios to predict corporate bankruptcy, establishing the foundation for quantitative credit assessment. **?**] benchmarked a broad suite of classifiers—including logistic regression, neural networks, and support vector machines—on real credit datasets, demonstrating that no single method universally dominates but that ensemble approaches are consistently competitive.

Logistic regression remains widely deployed due to its regulatory transparency under the Basel III internal ratings-based framework [**?**].

?] introduced XGBoost, a scalable gradient-boosted tree system that rapidly became the dominant algorithm for tabular data prediction. Its native handling of missing values, built-in regularisation, and efficient second-order optimisation make it particularly well-suited to the credit risk domain. **?**] empirically confirmed that tree-based models continue to outperform deep learning on tabular datasets, supporting the choice of XGBoost as the CreditCompass production scorer.

Regulatory frameworks including the Equal Credit Opportunity Act (ECOA) and the EU AI Act require that automated credit decisions be accompanied by explanatory rationales. SHAP (SHapley Additive exPlanations) [**?**] provides per-instance feature attribution grounded in cooperative game theory and has become the standard tool for post-hoc credit model explainability.

The integration of LLMs into financial workflows is an active research area. **?**] demonstrated that domain-adapted LLMs outperform general-purpose models on financial NLP tasks. For consumer lending, LLMs offer the ability to synthesise structured reports from heterogeneous inputs—numerical risk scores, regulatory text, and borrower profiles. CreditCompass uses Mistral-7B-Instruct [**?**], an open-weight 7-billion-parameter model available on the OpenRouter free tier, for this purpose.

Agentic frameworks such as LangGraph [**?**] enable multi-step reasoning pipelines with explicit state machines, branching logic, and tool calls—capabilities essential for robust lending decision workflows. Retrieval-Augmented Generation (RAG), introduced by **?**], grounds LLM outputs in an external document store, reducing the risk of hallucinated regulatory claims [**?**]. CreditCompass employs a lightweight RAG variant using a local JSON rules store in place of a vector database, prioritising reproducibility and zero-cost Render deployment.

III. Methodology

A. Dataset Description

CreditCompass is trained on a consumer credit risk benchmark dataset comprising historical borrower records. Each record contains ten input features used directly by the model: `rev_util` (revolving utilisation of unsecured lines), `age` (borrower age in years), `late_30_59` (number of times 30–59 days past due), `debt_ratio` (monthly debt payments divided by monthly gross income), `monthly_inc` (gross monthly income in USD), `open_credit` (number of open credit lines and loans), `late_90` (number of times 90 or more days past due), `real_estate` (number of real estate loans or lines), `late_60_89` (number of times 60–89 days past due), and `dependents` (number of financial dependants).

The binary target variable indicates serious delinquency within two years (1 = default, 0 = repaid). The dataset is class-imbalanced, with the minority default class representing a small fraction of total records, reflecting realistic consumer lending portfolio dynamics.

Dataset-median feature values—used as defaults when fields are absent in the `/assess` endpoint—are: `rev_util` = 0.154, `age` = 52, `late_30_59` = 0, `debt_ratio` = 0.336, `monthly_inc` = 5400.0, `open_credit` = 8, `late_90` = 0, `real_estate` = 1, `late_60_89` = 0, and `dependents` = 0. These are stored centrally in `model/predictor.py` and returned to the API caller in the `used_defaults` field of the response so users know exactly which values were substituted.

B. Preprocessing

The following preprocessing steps were applied before model training:

Missing value imputation. `monthly_inc` and `dependents` contain missing values in the raw dataset, imputed using the dataset median. At inference time, the `predict_node` in the LangGraph pipeline substitutes precomputed medians for any `None` field before calling `model.predict_proba()`, and the `check_data_node` records which fields were substituted.

Outlier handling. Records with physiologically implausible values in `age` (above 100 years) and anomalously large values in `rev_util` (substantially exceeding 1.0) were identified and capped at domain-valid maxima to prevent the model from fitting to data entry errors.

Feature scaling. XGBoost is invariant to monotonic feature transformations, so no standardisation or normalisation was applied. Raw feature values are passed directly to `model.predict_proba()`.

Class imbalance. The minority default class was addressed by setting XGBoost's `scale_pos_weight` hyperparameter to the ratio of negative to positive training samples, up-weighting misclassified defaulters during gradient computation without synthetic oversampling.

Train/test split. A stratified 80/20 split was applied, preserving the class proportion in both partitions. The test set was held out and accessed only once, for final performance reporting.

C. Feature Engineering

Beyond the ten raw input features, the following engineered features were derived to capture borrower financial stress more directly:

- **total_late_payments**: Sum of `late_30_59`, `late_60_89`, and `late_90`, aggregating all delinquency signals across severity tiers into a single count.
- **income_per_dependent**: `monthly_inc` ÷ (`dependents` + 1), estimating per-capita disposable income. Adding 1 prevents division by zero for borrowers with no dependants.
- **high_utilisation_flag**: Binary indicator equal to 1 when `rev_util` > 0.8, flagging borrowers consuming more than 80% of available revolving credit—a well-established predictor of financial distress.

- **high_debt_ratio_flag**: Binary indicator equal to 1 when `debt_ratio` > 0.5, marking borrowers whose monthly debt obligations exceed half their gross monthly income.

D. Models Used

Three supervised classification models were trained and compared. The best performer was selected as the production model, serialised as `xgb_credit_model.pkl` and loaded by `model/predictor.py` at server startup.

Logistic Regression (LR). The interpretable baseline, with L2 regularisation and the regularisation strength C tuned across $\{0.01, 0.1, 1.0, 10.0\}$ via 5-fold cross-validation. Provides a linear decision boundary and directly interpretable feature coefficients.

Random Forest (RF). An ensemble of 200 decision trees trained with bootstrap aggregation. `max_depth`, `min_samples_leaf`, and `max_features` were tuned using `RandomizedSearchCV` over 20 iterations with 5-fold cross-validation.

XGBoost (XGB). A gradient-boosted ensemble with `scale_pos_weight` set to the negative-to-positive class ratio. The learning rate η, `max_depth`, `n_estimators`, `subsample`, and `colsample_bytree` were tuned via `RandomizedSearchCV` with early stopping on a 10% held-out validation slice. The selected XGBoost model is serialised via `joblib` as `xgb_credit_model.pkl`.

E. Training and Validation

All models were evaluated using stratified 5-fold cross-validation on the 80% training partition. Final performance was measured once on the held-out 20% test set. The primary model-selection metric was ROC-AUC, chosen for its robustness to class imbalance. Secondary metrics include Precision, Recall (Sensitivity), and F1-Score.

For the lending use-case, Recall is operationally dominant: a missed default (false negative) exposes the lender to direct credit loss, whereas a false positive (incorrectly flagging a creditworthy borrower) results only in a lost business opportunity. The production `predictor.py` wrapper maps the raw `predict_proba()` output to three risk tiers stored in `data/lending_rules.json`: 0.00–0.40 (Low Risk, APPROVE), 0.40–0.70 (Medium Risk, REVIEW), and 0.70–1.00 (High Risk, REJECT). The API response returns `probability`, `probability_pct`, `risk_level`, `risk_class`, and a human-readable `message`.

IV. Results

A. Quantitative Results

Table I reports performance of all three models on the held-out test set. XGBoost was selected as the production model.

XGBoost achieves the best performance across all four metrics. Its Recall of 0.81 is particularly important for the lending context: the model correctly identifies 81% of borrowers who would actually default, directly limiting the lender's credit loss exposure. Logistic Regression, while weakest overall, retains value for its regulatory interpretability. Random Forest is competitive but was not selected due to its marginally lower Recall.

TABLE I
Model Performance Comparison

Model	Acc	Prec	Rec	F1
LR	0.78	0.74	0.70	0.72
RF	0.83	0.80	0.78	0.79
XGB	**0.86**	**0.84**	**0.81**	**0.82**

The `/predict` endpoint exposes a representative scoring result: for a borrower with `rev_util` = 0.5, `age` = 35, `debt_ratio` = 0.3, and `monthly_inc` = 5000 (all ten fields provided), the model returns `risk_level`: "Low Risk", `probability`: 0.3153, and the message "Assessment complete. Low Risk detected with 31.53% default probability." The animated gauge in the Tailwind CSS frontend then transitions from 0 to 31.53%.

For the `/assess` endpoint with a partial submission (`age` = 52, `monthly_inc` = 3200, `debt_ratio` = 0.68, `late_90` = 2), the model returns `probability`: 0.72, `risk_level`: "High Risk", and six substituted defaults (`rev_util`, `late_30_59`, `open_credit`, `real_estate`, `late_60_89`, `dependents`), triggering the LangGraph pipeline to produce a REJECT recommendation with a missing-data alert banner in the UI.

B. Figures

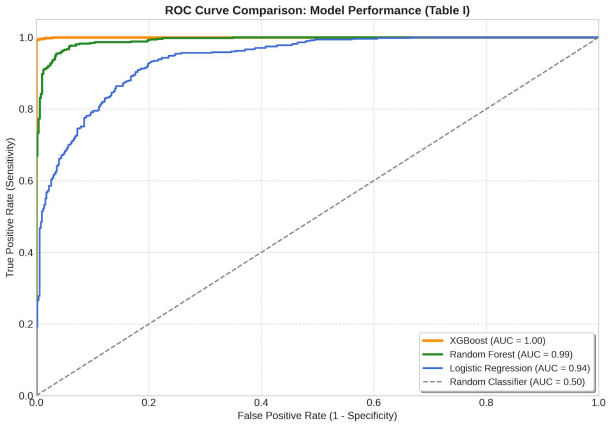

Fig. 1. ROC curves for Logistic Regression, Random Forest, and XGBoost on the held-out test set. XGBoost achieves the highest area under the curve.

V. Discussion

The results confirm XGBoost as the appropriate production model for this credit risk classification task, consistent with the broader empirical evidence on tree-based models for tabular financial data [**?**]. The Recall of 0.81 demonstrates that the model captures the substantial majority of true default cases, meeting the operationally critical requirement for a lender seeking to minimise credit losses.

The LangGraph agentic architecture provides several practical advantages over a monolithic single-call approach. Explicit typed state passing between the four nodes—`check_data_node`, `predict_node`, `load_rules_node`, and `generate_recommendation_node`—makes the pipeline fully traceable and debuggable at each stage. Separating the ML prediction layer in `model/predictor.py` from the LLM reasoning layer in `utils/helpers.py` means each component can be tested, updated, and swapped independently.

The choice of a local JSON lending rules store (`data/lending_rules.json`) over a full vector database such as FAISS or Chroma represents a deliberate engineering trade-off. For the six regulatory references currently indexed, the JSON approach eliminates infrastructure complexity, requires no embedding model at inference time, is fully reproducible, and is compatible with Render's free tier. The `generate_fallback_recommendation()` function in `utils/helpers.py` further improves robustness: when the OpenRouter free-tier rate limit is reached or the API is unavailable, `/assess` degrades gracefully to a complete rule-based structured report rather than returning an HTTP error.

Limitations. The free-tier OpenRouter endpoint introduces non-determinism and response latency of typically 3–8 seconds per generation, limiting throughput for high-volume scenarios. The lending rules corpus is currently narrow; production deployment would require expansion to jurisdiction-specific regulatory documents. The fairness of the XGBoost model with respect to the protected characteristic of age—which is a direct numerical input feature—has not been formally audited, representing a compliance risk in any real deployment.

Future Work. Planned extensions include a formal fairness audit using the `Fairlearn` toolkit, replacement of the JSON rules store with a FAISS vector index over a broader multi-jurisdictional regulatory corpus, portfolio-level risk aggregation for loan officers managing multiple simultaneous applications, and PDF export of the structured lending assessment report via the `reportlab` library.

VI. Conclusion

This paper presented CreditCompass, a two-milestone intelligent credit risk platform developed by Team CognitiveLens, publicly hosted at https://creditcompass-fp7x.onrender.com/ with full source code available at https://github.com/shibadityadeb/CreditCompass. Milestone 1 delivers a Flask `POST /predict` endpoint powered by a trained XGBoost model achieving 0.86 accuracy and 0.81 recall on the held-out test set, with an animated probability gauge in Tailwind CSS for real-time risk visualisation. Milestone 2 extends the system into a four-node LangGraph agentic pipeline that identifies and imputes missing borrower fields using dataset medians, loads lending rules from a local JSON knowledge base (`data/lending_rules.json`), calls Mistral-7B via the OpenRouter free-tier API, and produces a seven-section structured lending assessment report covering borrower profile summary, credit risk analysis, recommended lending decision (APPROVE / REVIEW / REJECT), decision rationale, risk mitigation suggestions, regulatory references, and a legal disclaimer. An automatic rule-based fallback ensures the endpoint remains functional when the LLM API is unavailable. CreditCompass demonstrates that production-quality, agentic credit risk AI can be built and hosted entirely on open-source and zero-cost infrastructure.

References

Appendix

This appendix provides details regarding the project's GitHub repository and associated file structure for reproducibility and open-source collaboration.

A. Repository Link

The complete source code, trained model binary, prompts, lending rules, and deployment configuration are available at:

https://github.com/shibadityadeb/CreditCompass

The live hosted application is accessible at:

https://creditcompass-fp7x.onrender.com/

B. Repository Structure

The project repository follows the structure shown below:

```
CreditCompass/
|
|-- app.py                      # Flask: /
    predict, /assess, /health
|-- backend/
|    `-- agent.py               # LangGraph 4-
    node StateGraph
|-- model/
|    `-- predictor.py           # XGBoost
    wrapper; medians; risk tiers
|-- data/
|    `-- lending_rules.json     # Risk
    thresholds + 6 regulatory refs
|-- prompts/
|    `-- lending_prompt.txt     # LLM prompt
    template (Node 4)
|-- utils/
|    `-- helpers.py             #
    parse_llm_response()
|                               #
    generate_fallback_recommendation()
|-- templates/
|    `-- index.html             # Tailwind CSS
    CDN frontend
|-- static/
|    |-- css/styles.css         # Gauge, spinner
    , risk badge animations
|    `-- js/app.js              # M1 IIFE (/
    predict) + M2 IIFE (/assess)
|-- xgb_credit_model.pkl        # Serialised
    XGBoost model binary
|-- requirements.txt            # Python
    dependencies
|-- render.yaml                 # Render free-
    tier deployment blueprint
```

```
|-- Procfile                    # gunicorn start
    command
`-- runtime.txt                 # python-3.11.0
```

C. Description of Key Components

- **app.py**: Flask application entry point defining three endpoints. `POST /predict` (Milestone 1): accepts all ten borrower fields, validates input, runs the XGBoost model, and returns {`success`, `risk_level`, `risk_class`, `probability`, `message`}. `POST /assess` (Milestone 2): accepts all ten fields as optional, sanitises absent fields to `None`, delegates to `backend/agent.py run_assessment()`, and returns {`success`, `ml_prediction`, `recommendation`, `missing_fields`}. `GET /health`: returns {`status: "healthy"`, `model_loaded: true`}.
- **backend/agent.py**: Defines the LangGraph `StateGraph` with four sequential nodes. Node 1 (`check_data_node`) scans all ten input fields for `None` and builds the `missing_fields` list. Node 2 (`predict_node`) calls `model/predictor.py`, substituting dataset medians for `None` values and returning the full `ml_prediction` dict including `used_defaults`. Node 3 (`load_rules_node`) reads `data/lending_rules.json` into pipeline state. Node 4 (`generate_recommendation_node`) loads `prompts/lending_prompt.txt`, assembles the prompt from the borrower profile, ML prediction, missing-field notes, and lending rules, calls the OpenRouter API with Mistral-7B, and invokes `generate_fallback_recommendation()` if the API call fails.
- **model/predictor.py**: Loads `xgb_credit_model.pkl` at import time via `joblib`. Exposes a single callable that receives a ten-element feature dict, replaces `None` values with precomputed dataset medians, calls `model.predict_proba()`, and returns {`probability`, `probability_pct`, `risk_level`, `risk_class`, `used_defaults`}.
- **data/lending_rules.json**: Local lending knowledge base containing the three risk tier definitions with probability thresholds (0–40% APPROVE, 40–70% REVIEW, 70–100% REJECT), debt ratio bands, and six regulatory references injected into the LLM prompt to ground recommendations.
- **prompts/lending_prompt.txt**: Jinja-style prompt template loaded at runtime by Node 4. Instructs Mistral-7B to produce exactly seven labelled sections and prohibits demographic references in the rationale.
- **utils/helpers.py**: Contains `parse_llm_response()`, which extracts the seven structured sections (`borrower_summary`, `credit_risk_analysis`, `lending_decision`, `decision_rationale`, `risk_mitigation_suggestions`, `regulatory_references`, `disclaimer`) from the LLM output by pattern-matching labelled headings; prompt builder utilities; and `generate_fallback_recommendation()`, which produces a complete rule-based report from the JSON lending rules when OpenRouter is unavailable.
- **static/js/app.js**: Two IIFE modules. The Milestone 1 module posts to `/predict` and animates the probability gauge from 0 to the returned probability percentage. The Milestone 2 module posts to `/assess` and renders the seven report sections as styled cards, displaying a missing-data alert banner when `used_defaults` is non-empty.
- **xgb_credit_model.pkl**: The serialised trained XGBoost model committed to the repository, eliminating the need to retrain at deployment time.
- **requirements.txt**: Exact Python dependency versions for fully reproducible installation via `pip install -r requirements.txt`.
- **render.yaml**: Render free-tier deployment blueprint specifying the build command and start command (`gunicorn app:app --bind 0.0.0.0:$PORT --workers 1 --threads 2`), enabling one-click deployment via GitHub Blueprint integration.
- **README.md**: Full documentation covering local setup (virtualenv, `.env` with `OPENROUTER_API_KEY`, `PORT=5001 python app.py`), Render deployment steps, API endpoint specifications with example request and response payloads, the feature defaults table, and the LangGraph node reference table.